Matrix factorization with Calepin

Matrix factorization represents a rectangular data matrix as the product of two thinner matrices. In recommender systems, the rows are users, the columns are items, and the observed entries are ratings. A rank k model writes

$$\hat{R}_{i,j} = \mu + u_i^T v_j$$

where u_i is a user vector, v_j is an item vector, and μ is the global average rating. The dot product is large when the user and item point in similar latent directions.[1]

[1] The sign and rotation of the latent dimensions are not identified. What matters for prediction is the dot product, not a unique interpretation of each axis.

A small ratings matrix

We will fit a rank-2 model to a tiny ratings matrix. Missing values are written as `None`.

```
import math
import random

users = ["Ada", "Ben", "Cy", "Dee", "Eli"]
items = ["Linear algebra", "Optimization", "Sci-fi", "Cooking"]

ratings = [
    [5.0, 4.0, None, 1.0],
    [4.0, None, None, 1.0],
    [1.0, 1.0, 5.0, 4.0],
    [None, 1.0, 4.0, 5.0],
    [5.0, None, 1.0, None],
]

observed = [
    (i, j, value)
    for i, row in enumerate(ratings)
    for j, value in enumerate(row)
    if value is not None
]

def show_matrix(matrix):
    print("          " + "  ".join(f"{name[:4]:>4}" for name in items))
    for name, row in zip(users, matrix):
        cells = ["   ." if value is None else f"{value:4.1f}" for value in row]
        print(f"{name:>4}      " + "  ".join(cells))

show_matrix(ratings)
```

```
          Line  Opti  Sci-  Cook
 Ada       5.0   4.0     .   1.0
 Ben       4.0     .     .   1.0
  Cy       1.0   1.0   5.0   4.0
 Dee         .   1.0   4.0   5.0
 Eli       5.0     .   1.0     .
```

The model should use the observed entries to fill in the missing cells. With only two latent dimensions, it has to compress the observed pattern rather than memorize every rating separately.

A synthetic rank-2 heatmap

Before fitting the ratings data, it is useful to look at an exactly rank-2 matrix. The following chunk imports NumPy, generates two skinny factors, and plots their product with Matplotlib.

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(7)
left = rng.normal(size=(18, 2))
right = rng.normal(size=(2, 14))
rank_two = left @ right

plt.figure(figsize=(6, 3.8))
image = plt.imshow(rank_two, aspect="auto", cmap="viridis")
plt.title("Synthetic rank-2 matrix")
```

```
plt.xlabel("column")
plt.ylabel("row")
plt.colorbar(image, fraction=0.046, pad=0.04, label="value")
plt.tight_layout()
```

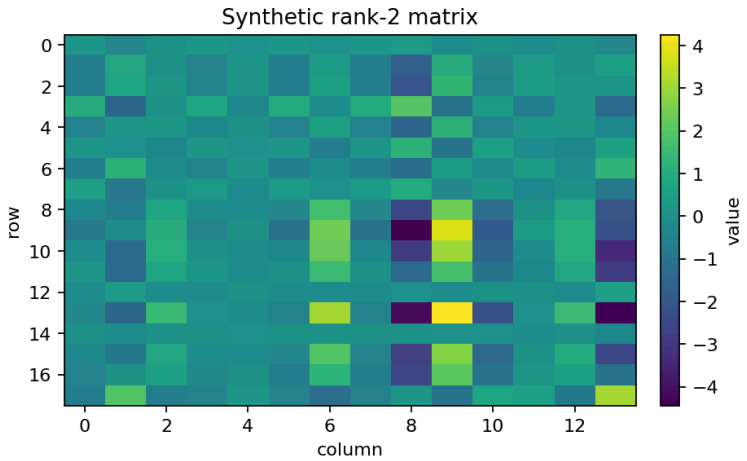


Figure 1: Synthetic rank-2 matrix

In HTML output, this figure is a useful asset-inlining check: a self-contained render should embed the generated PNG directly in the page instead of linking to a file under `.calepin/`.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus.

Fitting a rank-2 model

The loss is squared error on observed ratings plus a small penalty that keeps the latent vectors from growing too large:

$$L = \sum_{(i,j)\in\Omega} \left(r_{i,j} - \hat{R}_{i,j}\right)^2 + \lambda\left(\sum_i \|u_i\|^2 + \sum_j \|v_j\|^2\right)$$

Here is a simple stochastic gradient descent fit.

```
random.seed(12)

rank = 2
learning_rate = 0.025
regularization = 0.02
epochs = 2400

mu = sum(value for _, _, value in observed) / len(observed)
user_factors = [
    [random.uniform(-0.35, 0.35) for _ in range(rank)]
    for _ in users
]
item_factors = [
    [random.uniform(-0.35, 0.35) for _ in range(rank)]
    for _ in items
]

def dot(left, right):
    return sum(a * b for a, b in zip(left, right))

def predict(i, j):
    return mu + dot(user_factors[i], item_factors[j])

def rmse():
    total = 0.0
    for i, j, value in observed:
        total += (value - predict(i, j)) ** 2
    return math.sqrt(total / len(observed))

checkpoints = []
for epoch in range(epochs + 1):
    if epoch in (0, 25, 100, 400, 1200, 2400):
        checkpoints.append((epoch, rmse()))
    random.shuffle(observed)
    for i, j, value in observed:
        error = value - predict(i, j)
```

```
        old_user = user_factors[i][:]
        old_item = item_factors[j][:]
        for d in range(rank):
            user_factors[i][d] += learning_rate * (
                error * old_item[d] - regularization * old_user[d]
            )
            item_factors[j][d] += learning_rate * (
                error * old_user[d] - regularization * old_item[d]
            )

for epoch, value in checkpoints:
    print(f"epoch {epoch:4d}: observed RMSE = {value:.3f}")
```

```
epoch    0: observed RMSE = 1.755
epoch   25: observed RMSE = 0.463
epoch  100: observed RMSE = 0.270
epoch  400: observed RMSE = 0.127
epoch 1200: observed RMSE = 0.056
epoch 2400: observed RMSE = 0.056
```

Item	factor 1	factor 2
Linear algebra	1.41	−0.73
Optimization	1.37	−0.03
Sci-fi	−1.26	−0.86
Cooking	−0.73	1.27

Table 1: Learned item factors

The margin table is computed from the trained model. The exact coordinate system is arbitrary, but nearby items have similar factor vectors.

Completing the matrix

The fitted model can now predict the missing ratings.

```
completed = []
for i, row in enumerate(ratings):
    out = []
    for j, value in enumerate(row):
        out.append(value if value is not None else predict(i, j))
    completed.append(out)

show_matrix(completed)
```

```
          Line  Opti  Sci-  Cook
 Ada       5.0   4.0   3.0   1.0
 Ben       4.0   2.9   4.4   1.0
  Cy       1.0   1.0   5.0   4.0
 Dee       0.5   1.0   4.0   5.0
 Eli       5.0   5.0   1.0   2.1
```

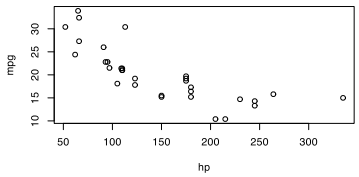
The same fitted object can also list only the originally missing entries.

```
for i, row in enumerate(ratings):
    for j, value in enumerate(row):
        if value is None:
            print(f"{users[i]:>3} -> {items[j]:<14} {predict(i, j):.2f}")
```

```
Ada -> Sci-fi         3.03
Ben -> Optimization   2.95
Ben -> Sci-fi         4.43
Dee -> Linear algebra 0.50
Eli -> Optimization   5.02
Eli -> Cooking        2.10
```

What to check

```
plot(mpg ~ hp, data = mtcars)
```



Matrix factorization is useful because it shares information across rows and columns. But this sharing is also a modeling assumption. A practical workflow should check prediction error on held-out observed entries, tune the rank k, and inspect whether the completed matrix makes domain sense. Calepin is useful here because the prose, the fitted model, and the diagnostics all live in the same executable document.